MORPH（ICSE 2027 Major Revision）论文修改前后英文原文及中文翻译

文档依据：以最新的 main_diff(1).pdf 为准（红色删除＝修改前，蓝色下划线＝修改后）；与上一版 main_diff.pdf 核对，并参考此前的 main.pdf 核实未变动文本。按 diff 稿的 PDF 页码排列。英文为相应修改点的原文摘录；为方便阅读，去除了 PDF 断行、恢复了字体拆散的 MORPH/SEDAR 等词。纯新增内容标记“原稿无对应段落”。数值、术语和结论依 PDF 保留，不主动改写。

本次更新（2026-11-03）：根据 main_diff(1).pdf 同步，修改项由 39 项增为 40 项；第 22 项为新版 RQ2 正文新增的等量种子实验总结（100/200/300 种子、24h×5、17.2%–92.4% 覆盖提升）。其余主要为断行、标记与分页位置变化，保留核对后仍有效的前后对照。

I. Abstract / Introduction（摘要与引言）

1. “Semantic Correctness” 指标更名（diff p.1，另见 p.2、p.7–12）

修改前英文： "MORPH covers 14% more branches, finds 16 more bugs in 24 hours, and improves the overall **semantic correctness** of transferred seeds by 130%."

修改前中文： MORPH 在 24 小时内覆盖的分支数增加了 14%，多发现 16 个漏洞，并使迁移后种子的整体**语义正确率**提高了 130%。

修改后英文： "MORPH covers 14% more branches, finds 16 more bugs in 24 hours, and improves the overall **execution validity** of transferred seeds by 130%."

修改后中文： MORPH 在 24 小时内覆盖的分支数增加了 14%，多发现 16 个漏洞，并使迁移后种子的整体**执行有效率**提高了 130%。

2. 对现有 SQL 翻译工作的概括更谨慎（diff p.1）

修改前英文： "Existing SQL translation techniques [15]–[17] primarily focus on statement-level translation, which treats SQL statements independently."

修改前中文： 现有 SQL 翻译技术 [15]–[17] 主要关注语句级翻译，即独立处理每条 SQL 语句。

修改后英文： "Existing SQL translation techniques [15], [16] primarily focus on statement-level translation."

修改后中文： 现有 SQL 翻译技术 [15]、[16] 主要关注语句级翻译。

3. 纠正 SEDAR comment-and-restore 机制的描述（diff p.1）

修改前英文： "(ii) Second, to satisfy strict grammar constraints imposed by downstream fuzzers, generated test cases are often sanitized by removing or commenting out unsupported constructs, which may simplify complex SQL structures and weaken execution behaviors."

修改前中文： （ii）其次，为满足下游模糊测试工具施加的严格语法约束，生成的测试用例通常通过删除或注释不支持的结构来进行清理。这可能简化复杂的 SQL 结构并削弱其执行行为。

修改后英文："(ii) Second, to satisfy strict grammar constraints imposed by downstream fuzzers, parser-incompatible fragments are handled after translation rather than resolved during it. For example, SEDAR temporarily comments them out during mutation and restores them before execution on the target DBMS, which retains the original query logic but leaves translation errors in these fragments unrepaired."

修改后中文：（ii）其次，为满足下游模糊测试工具施加的严格语法约束，与解析器不兼容的片段是在翻译之后进行处理，而不是在翻译过程中得到解决。例如，SEDAR 在变异期间暂时将这些片段注释掉，并在目标 DBMS 执行之前将其恢复。这种方式保留了原始查询逻辑，但没有修复这些片段中的翻译错误。

4. 修改总体覆盖率和漏洞增益区间（diff p.2 ）

修改前英文："Across the four DBMSs under 24 hours, MORPH achieves **14%–528%** higher branch coverage and discovers **16–22** more bugs than others."

修改前中文：在四种 DBMS 上进行 24 小时测试时，MORPH 的分支覆盖率比其他方法高 **14%–528%**，多发现 **16–22** 个漏洞。

修改后英文："Across the four DBMSs under 24 hours, MORPH achieves **7%–528%** higher branch coverage and discovers **5–22** more bugs than others."

修改后中文：在四种 DBMS 上进行 24 小时测试时，MORPH 的分支覆盖率比其他方法高 **7%–528%**，多发现 **5–22** 个漏洞。这里的下限变化主要与新增 SmartFuzz 等基线有关。

5. 第一项贡献从贬低 LLM 转为定义任务需求（diff p.2 ）

修改前英文（删除的核心表述）："LLM-based end-to-end translation often introduces errors and structural simplifications, and limited SQL grammar support further hinders mutation and degrades query semantics, reducing testing effectiveness."

修改前中文：基于 LLM 的端到端翻译经常引入错误和结构简化，而有限的 SQL 语法支持进一步阻碍变异、削弱查询语义，从而降低测试效果。

修改后英文："Transferring existing test cases requires coordinating cross-statement dependencies, target-side executability, and compatibility with downstream fuzzers."

修改后中文：迁移现有测试用例需要同时协调**跨语句依赖关系、目标端可执行性，以及与下游模糊测试工具的兼容性**。

6. Table I 能力分类及判断标准改写（diff p.2–3 ）

修改前英文："Cross-stmt. Deps. denotes explicit preservation of schema, data, and state across statements. ✓: explicit support; △: limited support; ×: not supported."

修改前中文：“跨语句依赖”表示显式保留不同语句之间的模式、数据和状态。✓ 表示明确支持；△ 表示有限支持；× 表示不支持。

修改后英文："Cross-stmt. Deps. denotes explicit tracking of forward read-after-write dependencies and schema/type context across statements. ✓: supported and documented or evaluated; △: partially supported, i.e., supported only for some statements or settings, or through a separate component; ×: not supported."

修改后中文： “跨语句依赖”表示显式跟踪语句之间的**前向读后写（RAW）依赖**以及模式／类型上下文。✓ 表示具备支持，且有文档说明或实验验证；△ 表示部分支持，即仅支持某些语句、特定配置，或通过独立组件实现；× 表示不支持。

表格结果变化：SQLGlot 的 Test Case Migration 调整为 △；jOOQ 的 Cross-stmt. Deps. 调整为 △；SEDAR 的 Cross-stmt. Deps. 调整为 △；CrackSQL 的 DML 标记在最新 diff 的目标版本中为 ✓。这些单元格以新版 main_diff(1).pdf 的 Table I 为准，而不是从删除线内容推断。

II. Background and Motivation（背景与动机）

7. SEDAR 与 MORPH 的核心差异被重新定义（diff p.3 ）

修改前英文： "SEDAR explores fuzzing-oriented seed transfer using LLMs. While it improves automation by translating complete test cases, it still does not explicitly preserve cross-statement dependencies during migration."

修改前中文： SEDAR 探索利用 LLM 开展面向模糊测试的种子迁移。尽管它通过翻译完整测试用例提高了自动化程度，但在迁移过程中仍未显式保留跨语句依赖。

修改后英文： "SEDAR transfers complete test cases, but it does not maintain translated target-side context across statements and leaves target-side execution-feedback regeneration as future work. MORPH instead explicitly represents test structures and forward read-after-write dependencies in an AQT, coordinates instantiation with evolving target-side schema and type context, and uses parser and execution feedback to repair transferred seeds before downstream mutation."

修改后中文： SEDAR 能够迁移完整测试用例，但它**不维护跨语句的翻译后目标端上下文**，并将基于目标端执行反馈的重新生成留作未来工作。相比之下，MORPH 在 AQT 中显式表示测试结构和前向读后写依赖，利用不断演化的目标端模式与类型上下文协调实例化过程，并在下游变异之前使用解析器和执行反馈修复迁移后的种子。

8. 动机示例：从“结构损失”改为“恢复后错误仍存在”（diff p.3 ）

修改前英文（关键论断）： "SEDAR comments out AGE and replaces the WHERE condition with True, producing a syntactically valid but simplified query that loses the original verification intent and fails to exercise deep internal states. As a result, the migrated seed has limited utility for subsequent effective mutation in DBMS fuzzing."

修改前中文： SEDAR 将 AGE 注释掉，并将 WHERE 条件替换为 True，从而生成语法正确但经过简化的查询。这会丢失原始验证意图，无法触达深层内部状态。因此，该迁移种子对后续有效的 DBMS 模糊测试变异作用有限。

修改后英文： "In this example, the LLM retains AGE in the translated predicate. To satisfy the fuzzer's parser, SEDAR temporarily comments out this fragment and replaces the predicate with TRUE during mutation, then restores the fragment before execution. Since the restored fragment still contains the translation error, the mutated test case fails on DuckDB."

修改后中文： 在这个示例中，LLM 在翻译后的谓词中仍保留 AGE。为了满足模糊测试工具的解析器要求，SEDAR 在变异期间暂时注释该片段，并将谓词替换为 TRUE，随后在执行前恢复该片段。由于恢复后的片段仍然包含翻译错误，变异得到的测试用例在 DuckDB 上执行失败。

9. 动机图 Fig. 1 的说明文字（diff p.3 ）

修改前英文（核心措辞）： "Conventional end-to-end translation methods like SEDAR suffer from semantic error and structural simplification."

修改前中文： SEDAR 等传统端到端翻译方法存在语义错误和结构简化问题。

修改后英文（核心措辞）： "Motivating example observed when running SEDAR on a PostgreSQL seed (Left): the incompatible fragment is commented out during mutation and restored before execution, leaving the translation error unrepaired."

修改后中文： 在 PostgreSQL 种子上运行 SEDAR 时观察到的动机示例（左）：不兼容片段在变异期间被注释，并在执行前恢复，因此翻译错误未被修复。

10. Basic Idea：去掉"保证高保真语义"的夸大表述（ diff p.3 ）

修改前英文（删除的关键论断）： "By maintaining these complex semantics during transfer, MORPH ensures the generated seeds are effective for validation, enabling them to reach deeper code paths within the DBMS and uncover intricate logic bugs that might be unreachable by SEDAR."

修改前中文： 通过在迁移过程中保留这些复杂语义，MORPH 确保生成的种子适合有效性验证，使其能到达 DBMS 更深层的代码路径，并发现 SEDAR 可能无法触及的复杂逻辑漏洞。

修改后英文： "For multi-statement test cases, it propagates target-side schema and type context in statement order and uses this context together with target-dialect information to instantiate AQT nodes. Parser and execution feedback guide the repair of remaining incompatibilities before mutation. In this example, MORPH instantiates <TimeDiff> with date_diff, producing an executable seed that retains the temporal predicate."

修改后中文： 对于多语句测试用例，MORPH 按语句顺序传播目标端模式与类型上下文，并结合这些上下文及目标方言信息来实例化 AQT 节点。解析器和执行反馈用于指导变异前对剩余不兼容问题的修复。在本示例中，MORPH 将 `<TimeDiff>` 实例化为 `date_diff`，生成保留时间谓词且可执行的种子。

III. Approach（方法）

11. Object-state summaries 的内容（diff p.4 ）

修改前英文： 原稿只有相邻的图定义描述，未明确列出 object-state summary 的组成；对应定义中将依赖图称为 "for dependency-preserving ordering"。

修改前中文： 原稿未具体定义对象状态摘要的字段，并将依赖图的用途描述为“用于保持依赖关系的排序”。

修改后英文： "Each object-state summary records whether a schema object exists, its table definition, column names and types, constraints, and the schema changes induced by preceding DDL statements; the summaries are updated incrementally in statement order."

修改后中文： 每个对象状态摘要记录一个模式对象是否存在、其表定义、列名与类型、约束，以及此前 DDL 语句引起的模式变化；这些摘要按照语句顺序进行增量更新。

12. 依赖图的作用：排序 → 选择相关上下文（diff p.4–5 ）

修改前英文： "Gtc is its graph representation for dependency-preserving ordering." / "The instantiation respects the dependency order in Gtc: DDL and DML statements are instantiated before queries that depend on them."

修改前中文： Gtc 是用于保留依赖顺序的图表示。实例化过程遵循 Gtc 指示的依赖顺序：DDL 与 DML 语句先于依赖它们的查询被实例化。

修改后英文： "Gtc is its graph representation, used to select dependency-relevant context during instantiation." / "The instantiation follows the original statement order. The dependency edges in Gtc identify dependency-relevant preceding statements whose target-side context is used during instantiation and repair."

修改后中文： Gtc 是依赖集合的图表示，用于在实例化期间**选择依赖相关的上下文**。实例化仍遵循原始语句顺序；Gtc 中的依赖边识别与当前语句有关的先前语句，并在实例化和修复时使用它们对应的目标端上下文。

13. Algorithm 2：删去稳定拓扑排序，明确 SelectContext（diff p.5 ）

修改前英文： "The algorithm first computes a dependency-preserving statement order over Gtc while keeping the original order for independent statements." 算法原先包含 `StmtOrder ← STABLETOPOLOGICALSORT(Gtc, ⟨1, ..., |U|⟩)`。

修改前中文： 算法首先根据 Gtc 计算保持依赖关系的语句顺序，并保持相互独立的语句原有顺序。伪代码原先显式调用稳定拓扑排序。

修改后英文： "At the outer level, the algorithm processes statements in their original order. For each statement si, SELECTCONTEXT uses the dependency set Δ to select its direct predecessors {j | (j, i) ∈ Δ}. Among the successfully instantiated predecessors, it keeps only the most recent one for each set of objects shared with si, since earlier writes to the same objects are superseded, and retains at most five predecessors."

修改后中文： 在外层，算法按原始顺序处理语句。对于每条语句 si，`SELECTCONTEXT` 根据依赖集合 Δ 选出其直接前驱 `{j | (j, i) ∈ Δ}`。在成功实例化的前驱中，对于与 si 共享的每组对象，仅保留最近的一条相关前驱，因为针对相同对象的更早写操作已被后续写操作覆盖；最终最多保留五条前驱语句。

14. “Guarantee semantic consistency” 改成 “Keep valid”（diff p.5 ）

修改前英文： "To guarantee that the generated queries remain semantically consistent with the target dialect during test-case instantiation ..."

修改前中文： 为了保证生成的查询在测试用例实例化期间与目标方言保持语义一致……

修改后英文： "To keep the generated queries valid in the target dialect during test-case instantiation ..."

修改后中文： 为了使生成的查询在测试用例实例化期间**在目标方言中保持有效**……

15. 最后一级修复的风险被明确说明（diff p.6 ）

修改前英文： "Thus, destructive commenting is used only after structure-preserving repairs fail."

修改前中文： 因此，只有在保留结构的修复失败后才使用破坏性注释。

修改后英文："Thus, only if the preceding three fallback stages fail does MORPH comment out the minimal offending fragment. Constant substitution and minimal commenting may change the behavior of transferred seeds."

修改后中文：因此，只有当前面三个回退阶段全部失败时，MORPH 才会注释掉最小的出错片段。常量替换和最小化注释可能改变迁移后种子的行为。

IV. Evaluation（实验评估）

16. 新增四库种子数量说明（diff p.7 ）

修改前英文：原稿在所对应位置仅说明收集了 "6,875 test files, totaling 55.7 MiB of query text."，没有紧随其后的四库种子数说明。

修改前中文：原稿说明共收集 6,875 个测试文件、查询文本总计 55.7 MiB，但没有在该处明确列出各库使用的种子数量。

修改后英文："For each target DBMS, all methods use the same number of initial seeds, i.e., 5,879, 4,551, 5,850, and 5,990 seeds for MonetDB, Virtuoso, DuckDB, and ClickHouse, respectively."

修改后中文：对于每个目标 DBMS，所有方法使用相同数量的初始种子，分别为 MonetDB 5,879 个、Virtuoso 4,551 个、DuckDB 5,850 个和 ClickHouse 5,990 个。

17. Execution Validity 的操作性定义（diff p.7 ）

修改前英文："Semantic Correctness, defined as the proportion of generated seeds that execute without runtime errors. Here, semantic correctness is defined by executability in the target DBMS without runtime errors, not by result equivalence between the source and target DBMSs."

修改前中文：“语义正确率”定义为能够在不发生运行时错误的情况下执行的种子比例。这里的语义正确指在目标 DBMS 上无运行时错误地执行，而不是源端与目标端结果等价。

修改后英文："Execution Validity, defined as the proportion of generated seeds that execute without runtime errors. Here, execution validity is defined by executability in the target DBMS without runtime errors, not by result equivalence between the source and target DBMSs."

修改后中文：“执行有效率”定义为能够在不发生运行时错误的情况下执行的种子比例。这里的执行有效性是指在目标 DBMS 上无运行时错误地执行，**不表示源端与目标端的结果等价**。

18. RQ1 完整漏洞发现活动时长（diff p.7 ）

修改前英文：原稿在此处只描述在四个目标 DBMS 上执行漏洞发现活动，未明确每库时长。

修改前中文：原稿没有在该段明确交代完整漏洞发现活动的持续时间。

修改后英文（新增部分）："for seven days on each target DBMS"

修改后中文：在每个目标 DBMS 上分别持续运行七天。

19. RQ2 扩展基线工具（diff p.8 ）

修改前英文："We compared MORPH against four state-of-the-art DBMS testing tools in 24 hours: SQLancer, SQLRight, SQUIRREL, GRIFFIN, and SEDAR."

修改前中文：我们在 24 小时内将 MORPH 与四种先进的 DBMS 测试工具进行比较：SQLancer、SQLRight、SQUIRREL、GRIFFIN 和 SEDAR。（旧稿此处“four”与实际列名个数存在不一致。）

修改后英文（修改的关键部分）："We compared MORPH against six state-of-the-art DBMS testing tools in 24 hours: SQLancer, SQLRight (coverage-guided logic-bug fuzzer), SQUIRREL, GRIFFIN, SEDAR, and SmartFuzz (LLM-based seed generation)."

修改后中文：我们在 24 小时内将 MORPH 与六种先进的 DBMS 测试工具进行比较：SQLancer、SQLRight（覆盖率引导的逻辑漏洞模糊测试工具）、SQUIRREL、GRIFFIN、SEDAR 和 SmartFuzz（基于 LLM 的种子生成方法）。

20. RQ2 新增置信区间和显著性说明（diff p.8 ）

修改前英文：原稿此处只有分支覆盖率对比和覆盖增长趋势分析，没有紧接着给出统计检验结果。

修改前中文：原稿主要呈现覆盖数量和增长曲线，缺少该段新增的置信区间和统计显著性报告。

修改后英文："Figure 7 shows the mean coverage and its 95% confidence interval over five runs, and all improvements of MORPH over the baselines are statistically significant (Mann–Whitney U test, $p < 0.05$)."

修改后中文：图 7 展示了五次运行的平均覆盖率及其 95% 置信区间；MORPH 相对基线的覆盖率提升均具有统计显著性（Mann–Whitney U 检验，$p < 0.05$）。

21. 旧稿对覆盖提升的强因果解释被删去（diff p.8 ）

修改前英文："This proves that coverage gains are not merely statistical but functionally impactful, enabling the detection of bugs that remain unreachable to baselines. Unlike SEDAR, which sacrifices semantic depth through destructive sanitization, MORPH employs structure-aware transfer to retain the full logical complexity of source test cases."

修改前中文：这证明覆盖率提升不只是统计意义上的差异，还具有实际功能效果，使工具能够发现基线无法触及的漏洞。与通过破坏性清理牺牲语义深度的 SEDAR 不同，MORPH 采用结构感知迁移保留源测试用例完整的逻辑复杂度。

修改后英文（替换的新论述）："Bug counts are the union of unique bugs across the five runs after stack-trace deduplication and manual verification. We further pair existing translators with the same downstream fuzzer on DuckDB: MORPH covers 33.6% and 16.7% more branches than SQLGlot+GRIFFIN and RISE+GRIFFIN, respectively, and detects 3 more bugs than each."

修改后中文：漏洞数量为五次运行中发现的独立漏洞的并集，经过调用栈去重和人工验证。我们还在 DuckDB 上让不同翻译工具使用相同的下游模糊测试工具：MORPH 的分支覆盖数分别比 SQLGlot+GRIFFIN 和 RISE+GRIFFIN 高 33.6% 和 16.7%，并且比两者各多发现 3 个漏洞。

22. 新版 RQ2 正文补充等量种子实验的总结（diff p.8；相上一版新增 ）

修改前英文：原始投稿稿在此处无对应段落；上一版 main_diff.pdf 的修改后正文也没有这一段集中总结。

修改前中文：原稿没有在 RQ2 的这处正文中总结等量种子下的覆盖率增益；该段为最新版 diff 新增。

修改后英文："To control for seed quantity, we also randomly sample 100, 200, and 300 seeds generated by MORPH and SEDAR as the initial seeds of GRIFFIN (24 hours, five runs). Under the same seed count, MORPH achieves 17.2%–92.4% higher coverage than SEDAR across the four DBMSs, all exceeding the 14% gain with the full seed corpora, which indicates that the improvement does not stem from a larger seed corpus."

修改后中文：为控制种子数量的影响，我们还分别从 MORPH 和 SEDAR 生成的种子中随机抽取 100、200 和 300 个作为 GRIFFIN 的初始种子（每组运行 24 小时，重复五次）。在种子数量相同的条件下，MORPH 在四种 DBMS 上的覆盖率比 SEDAR 高 17.2%–92.4%，均超过完整种子语料条件下 14% 的提升幅度。这表明 MORPH 的覆盖率提升并非源于更大的种子语料。

23. SmartFuzz 互补实验（diff p.8–9，纯新增）

修改前英文：无对应段落。

修改前中文：原稿没有 SmartFuzz 与 MORPH 合并种子的互补实验。

修改后英文："We further run GRIFFIN with the union of their seeds, which covers 413,021 branches and detects 35 bugs in total. SmartFuzz and MORPH share only 5 bugs, and each detects bugs that the other misses (12 by SmartFuzz and 17 by MORPH). The combination further detects 6 bugs (4 on MonetDB and 2 on Virtuoso) that neither SmartFuzz nor MORPH detects alone. However, its coverage is lower than that of MORPH alone on MonetDB and DuckDB, where the additional seeds reduce the execution throughput, leaving fewer mutation opportunities for potentially more valuable seeds."

修改后中文：我们进一步将 SmartFuzz 与 MORPH 的种子集合并，并使用 GRIFFIN 进行测试，总共覆盖 413,021 个分支、发现 35 个漏洞。SmartFuzz 和 MORPH 仅共享 5 个漏洞，且两者分别发现了另一方未发现的漏洞（SmartFuzz 独有 12 个，MORPH 独有 17 个）。合并种子又发现了二者单独运行均未发现的 6 个漏洞（MonetDB 上 4 个、Virtuoso 上 2 个）。不过，在 MonetDB 和 DuckDB 上，合并后的覆盖率低于单独使用 MORPH 时的覆盖率，因为额外种子降低了执行吞吐量，使潜在高价值种子获得的变异机会减少。

24. Mutation-handling 受控比较（diff p.9，纯新增）

修改前英文：无对应段落。

修改前中文：原稿没有固定翻译种子、仅改变变异处理策略的受控实验。

修改后英文："MORPH repairs incompatibilities before mutation, whereas SEDAR uses comment-and-restore handling (Section II). Neither strategy guarantees semantic equivalence of the mutated queries. For each target DBMS, we randomly sampled 300 translated seeds and applied both strategies to the same seeds. Each configuration was run with GRIFFIN for 24 hours and repeated five times. The translated inputs, target database initial state, GRIFFIN version and configuration, and resource allocation were held fixed; only the mutation-handling strategy differed. Across the four DBMSs, MORPH's handling achieves 27.2%, 15.5%, 4.0%, and 9.4% higher branch coverage than SEDAR's handling on DuckDB, MonetDB, Virtuoso, and ClickHouse, respectively."

修改后中文：MORPH 在变异前修复不兼容问题，而 SEDAR 采用注释后恢复的处理策略（见第二节）。两种策略都不能保证变异后查询的语义等价性。对于每个目标 DBMS，我们随机选取 300 个迁移种子，并对同一批种子应用两种处理策略。每组配置均使用 GRIFFIN 运行 24 小时，独立重复五次。翻译输入、目标数据库初始状态、GRIFFIN 版本与配置、资源分配均保持不变，唯一差异是变异处理策略。在 DuckDB、MonetDB、Virtuoso 和 ClickHouse 上，MORPH 策略的分支覆盖率分别比 SEDAR 策略高 27.2%、15.5%、4.0% 和 9.4%。

25. SEDAR 二进制的来源与可复现性（diff p.9 ）

修改前英文："We reproduced SEDAR's complete pipeline, including schema capture, end-to-end LLM translation, and its comment-out sanitization strategy."

修改前中文：我们复现了 SEDAR 的完整流程，包括模式捕获、端到端 LLM 翻译以及注释式清理策略。

修改后英文："We used the original SEDAR binary obtained from the authors of SEDAR, which runs its complete pipeline (Section II). Our artifact includes the SEDAR configuration and the SEDAR-generated seeds used in our experiments."

修改后中文：我们使用了从 SEDAR 作者处获取的原始 SEDAR 二进制程序，该程序运行其完整流程（见第二节）。我们的 Artifact 包含实验所使用的 SEDAR 配置以及 SEDAR 生成的种子。

26. 翻译基线的目标方言适配方式（diff p.9，新增）

修改前英文：该段原本未明确提供下面这些配置与输入公平性细节。

修改前中文：原稿缺少目标方言知识来源、逐语句翻译策略和无额外人工规则的说明。

修改后英文："Each baseline follows its original translation workflow, using the official SQL documentation of each target DBMS as dialect knowledge when required. CrackSQL and RISE translate each statement individually without schema context, and all methods use the same source test cases without additional manually written translation rules."

修改后中文：每个基线均遵循其原有翻译工作流程，并在需要时使用目标 DBMS 的官方 SQL 文档作为方言知识。CrackSQL 和 RISE 不使用模式上下文，逐条翻译每条语句；所有方法使用相同的源测试用例，且不引入额外人工编写的翻译规则。

27. 新增源端与目标端结果／状态等价性测试（diff p.9–10 ）

修改前英文："The evaluation targets fuzzing seed quality rather than cross-DBMS result equivalence."

修改前中文：本评估关注模糊测试种子质量，而不是跨 DBMS 的结果等价性。

修改后英文："The primary evaluation targets fuzzing seed quality: valid seeds must execute on the target DBMS, and rich seeds should retain diverse SQL features for mutation. The supplementary study separately assesses result and state equivalence."

修改后中文：主要评估关注模糊测试种子质量：有效种子应能在目标 DBMS 上执行，而特征丰富的种子应保留多样化 SQL 特征供后续变异。补充实验则独立评估**查询结果等价性**和**数据库状态等价性**。

新增英文："We randomly sampled 100 MySQL test cases and used the same cases to evaluate SQLGlot, RISE, SEDAR, and MORPH for transfer to DuckDB. We compared query results between the source and target executions for result equivalence, and compared the resulting database states, including schemas and table contents, for state equivalence. MORPH achieves 52.0% result equivalence and 74.0% state equivalence, compared with 14.0%/56.0% for SQLGlot, 34.0%/65.0% for RISE, and 38.0%/69.0% for SEDAR."

新增中文：我们随机抽取了 100 个 MySQL 测试用例，用同一批用例评估 SQLGlot、RISE、SEDAR 和 MORPH 向 DuckDB 的迁移结果。通过比较源端和目标端执行结果判断结果等价性，并通过比较包括模式和表内容在内的数据库最终状态判断状态等价性。MORPH 的结果等价率为 52.0%，状态等价率为 74.0%；SQLGlot、RISE 和 SEDAR 分别为 14.0%/56.0%、34.0%/65.0% 和 38.0%/69.0%。

28. 修正 RQ3 中一处数值（diff p.9 ）

修改前英文："... improves semantic correctness from 0.314 to **0.776**."

修改前中文：……将语义正确率从 0.314 提升至 **0.776**。

修改后英文："... improves execution validity from 0.314 to **0.748**."

修改后中文：……将执行有效率从 0.314 提升至 **0.748**。

29. SQL 特征丰富度结论降调（diff p.10 ）

修改前英文（删除论断）："The diversity gap mainly comes from how each baseline handles unmappable constructs: statement-level translators raise errors or omit them, while SEDAR comments them out, whereas MORPH constructively instantiates abstract structures into target-compliant forms via retrieved dialect evidence, preserving more feature-rich inputs for mutation."

修改前中文：多样性差距主要来自各基线处理无法映射的结构的方式：语句级翻译器报错或省略这些结构，SEDAR 将其注释掉，而 MORPH 则借助检索到的方言证据将抽象结构实例化为符合目标方言的形式，从而保留更多丰富特征以供变异。

修改后英文："These counts show that MORPH produces transferred seeds with richer SQL features. Its dialect-guided AQT instantiation and feedback repair aim to adapt complex structures to the target DBMS and downstream fuzzer."

修改后中文：这些计数结果表明，MORPH 生成的迁移种子具有更加丰富的 SQL 特征。其方言引导的 AQT 实例化和反馈修复旨在使复杂结构适配目标 DBMS 与下游模糊测试工具。

30. 消融实验从单一配对扩展至所有配对（diff p.10–11 ）

修改前英文（原始关键结论）："When both the Structure-Preserving Abstraction and the Dialect-Aware AQT Instantiation are disabled, the syntactic correctness is 51% and the semantic correctness of MORPHbase is 27%. ... The Full framework achieves the optimal balance, reaching 95% syntactic and 65% semantic correctness."

修改前中文：当结构保持抽象和方言感知 AQT 实例化均被关闭时，语法正确率为 51%，MORPHbase 的语义正确率为 27%。……完整框架实现了最佳平衡，语法正确率达 95%，语义正确率达 65%。

修改后英文（核心结果）："We evaluated four main variants on all source-target pairs used in RQ3. ... MORPHbase achieves 43.1% syntactic correctness and 35.2% execution validity. ... MORPHabs raises it to 73.6% ... and raises execution validity to 55.0%. MORPHrag raises [execution validity] to 65.5% ... and raises syntactic correctness to 63.9%. Combining both components, MORPHfull achieves the highest syntactic correctness (87.1%) and execution validity (71.6%)."

修改后中文：我们在 RQ3 使用的全部源端—目标端配对上评估了四种主要变体。MORPHbase 的语法正确率为 43.1%，执行有效率为 35.2%；MORPHabs 将语法正确率提高到 73.6%，执行有效率提高到 55.0%；MORPHrag 将执行有效率提高到 65.5%，语法正确率提高到 63.9%。结合两个组件后，MORPHfull 取得最高的语法正确率 87.1% 和执行有效率 71.6%。

31. 上下文传播消融的结果更新（ diff p.11 ）

修改前英文："This reduces semantic correctness from 65.1% to 52.4% (-12.7%) while only slightly reducing syntactic correctness to 93.5% (-1.7%)."

修改前中文：这使语义正确率从 65.1% 下降到 52.4%（下降 12.7 个百分点），同时仅使语法正确率小幅下降到 93.5%（下降 1.7 个百分点）。

修改后英文："This reduces execution validity from 0.716 to 0.523 (Table VII)."

修改后中文：这使执行有效率从 0.716 降到 0.523（见表 VII）。

32. 新增依赖图的独立消融（diff p.11 ）

修改前英文：无对应段落。

修改前中文：原稿没有将“依赖图带来的上下文选择收益”与“目标端上下文传播收益”分离评估。

修改后英文："To isolate the dependency graph from context propagation, we compare MORPHfull with MORPHwin, which uses the five most recent statements with the same object summaries, and MORPHall, which provides all preceding statements and the complete context. As shown in Table VII, MORPHfull achieves the highest execution validity (0.716 vs. 0.599 and 0.643) with 11.9% and 72.3% fewer input tokens per statement, whereas only 22.6% and 4.2% of the predecessors provided by MORPHwin and MORPHall are dependency-relevant. Thus, the dependency graph provides a compact, dependency-relevant context."

修改后中文：为将依赖图的作用与上下文传播分离，我们比较 MORPHfull、MORPHwin 和 MORPHall。MORPHwin 使用最近的五条语句以及相同的对象状态摘要；MORPHall 提供所有先前语句和完整上下文。表 VII 显示，MORPHfull 的执行有效率最高（0.716，对比 0.599 和 0.643），同时每条语句的输入 token 数分别减少 11.9% 和 72.3%；而 MORPHwin 和 MORPHall 提供的前驱语句中，真正与依赖相关的仅占 22.6% 和 4.2%。因此，依赖图能够提供紧凑且与依赖相关的上下文。

V. Discussion（讨论／工程开销）

33. 增加迁移开销与 SEDAR 对比（diff p.12 ）

修改前英文：在该段未明确给出全语料的总费用与 SEDAR 的逐种子成本对比。

修改前中文：原稿缺少此次新增的全量处理成本统计。

修改后英文："Processing all test files (65.1 statements per test suite on average) with Gemini 2.5-Flash-Lite costs $96 in total, i.e., $0.004 and 219 seconds per seed, compared with $0.006 and 146 seconds per seed and a total cost of $144 for SEDAR."

修改后中文：使用 Gemini 2.5-Flash-Lite 处理全部测试文件（平均每个测试套件包含 65.1 条语句）的总费用为 96 美元，即每个种子约 0.004 美元、219 秒。相比之下，SEDAR 每个种子约需 0.006 美元、146 秒，总费用为 144 美元。

34. 补充自动生成规则的人工修正流程（diff p.12 ）

修改前英文：原稿相邻文字只提及规则审查和修正，没有新增的明确验证步骤。

修改前中文：原稿没有具体说明如何确认自动生成规则是否正确。

修改后英文："To validate the generated rules, we apply them to translated test cases and check the resulting translations for errors; rules that produce incorrect translations are manually inspected and corrected."

修改后中文：为验证生成规则，我们将其应用于迁移后的测试用例，并检查产生的翻译是否存在错误；对于导致错误翻译的规则，则进行人工检查和修正。

VI. Related Work（相关工作）

35. 新增 SQLRight、sqllogictest 和种子调度工作（diff p.12 ）

修改前英文：在相应位置无这些完整介绍。

修改前中文：原稿对这些方向的介绍不足。

修改后英文："SQLRight combines coverage-guided mutation with logic-bug oracles. sqllogictest validates query results across database engines with a shared test format. Seed-selection and scheduling techniques, such as corpus minimization and power schedules, choose or prioritize seeds within an existing corpus, whereas MORPH constructs the initial corpus for target DBMSs that lack one; the two directions are orthogonal and can be combined."

修改后中文：SQLRight 将覆盖率引导的变异与逻辑漏洞测试预言机相结合。sqllogictest 使用统一的测试格式对不同数据库引擎的查询结果进行验证。种子选择和调度技术（如语料最小化和能量调度）在已有语料库中选择或优先执行种子，而 MORPH 为缺乏初始语料的目标 DBMS 构建初始语料。两类研究方向相互正交，可以结合使用。

36. 加入 SmartFuzz 并明确互补关系（diff p.12，纯新增）

修改前英文：无对应段落。

修改前中文：原稿没有在相关工作中充分讨论 SmartFuzz。

修改后英文："SmartFuzz uses LLMs to synthesize new seeds from features extracted from official DBMS documentation and historical bug-triggering inputs, targeting bug-prone execution states. In contrast, MORPH transfers existing test cases from mature DBMSs, which cover a broad range of SQL features, and preserves their cross-statement structures. The two approaches therefore provide complementary seeds: SmartFuzz targets bug-prone features, whereas MORPH provides broad feature coverage, and they could be combined to construct initial seeds for DBMS fuzzing."

修改后中文：SmartFuzz 利用 LLM，根据 DBMS 官方文档及历史漏洞触发输入中提取的特征合成新种子，重点覆盖容易触发漏洞的执行状态。相比之下，MORPH 从成熟 DBMS 迁移已有测试用例，这些用例涵盖较广的 SQL 特性，同时保留跨语句结构。因此，这两种方法提供的是互补性种子：SmartFuzz 关注容易触发漏洞的特性，MORPH 提供广泛的特征覆盖；二者可以结合，用于构建 DBMS 模糊测试的初始种子。

37. SEDAR 与 MORPH 的创新定位（diff p.12 ）

修改前英文："However, these methods largely rely on direct text generation or post-generation filtering and sanitization. In contrast, MORPH performs AQT-guided whole-test-case instantiation, where explicit schema dependencies, nested structures, and clause-level relationships support local repair while preserving context."

修改前中文：然而，这些方法主要依赖直接文本生成，或生成后的过滤与清理。相比之下，MORPH 执行 AQT 引导的完整测试用例实例化，利用显式模式依赖、嵌套结构和子句关系支持局部修复，并保留上下文。

修改后英文："SEDAR is the closest work to MORPH because it also transfers existing DBMS test cases into fuzzing seeds; its compatibility mechanism is described in Section II. Unlike SEDAR, which translates each statement of a test case individually under its referenced source-side sub-schema, MORPH performs AQT-guided instantiation that maintains target-side context across statements, where explicit dependencies, nested structures, and clause-level relationships support local repair with parser and execution feedback before mutation."

修改后中文：SEDAR 是与 MORPH 最接近的工作，因为它同样将现有 DBMS 测试用例迁移为模糊测试种子；其兼容性处理机制已在第二节说明。SEDAR 使用每条语句所引用的源端子模式作为上下文，逐条翻译测试用例中的语句。不同的是，MORPH 通过 AQT 引导的实例化维护跨语句的目标端上下文，并结合显式依赖、嵌套结构及子句级关系，在变异前利用解析器和执行反馈进行局部修复。

38. 纠正 CrackSQL 与 RISE 的描述（diff p.12–13 ）

修改前英文："CrackSQL employs genetic algorithms alongside LLMs to mine complex mutation patterns." / "RISE translates a query by first reducing a complex input ... and then applying [the rewrite] back to the original query."

修改前中文：CrackSQL 结合遗传算法和 LLM 挖掘复杂变异模式。RISE 通过先简化复杂输入、学习规则，再将重写应用到原始查询来完成翻译。

修改后英文："CrackSQL combines functionality-based query processing, cross-dialect syntax embedding with contrastive learning, and local-to-global translation." / "RISE reduces a complex query or procedural artifact to a simpler form, learns the local dialect rewrite from that reduced form, and applies the rewrite back to the original artifact."

修改后中文：CrackSQL 结合基于功能的查询处理、利用对比学习的跨方言语法嵌入，以及从局部到整体的翻译。RISE 将复杂查询或过程式程序单元简化为较简单的形式，从简化形式中学习局部方言重写规则，再将该规则应用回原始程序单元。

39. 明确 RISE、QTRAN 与 MORPH 的任务差异（diff p.13 ）

修改前英文："Overall, existing SQL translation methods treat a single query as the translation unit rather than targeting fuzzing-oriented whole-test-case seed transfer where cross-statement dependencies, evolving schema context, and compatibility with downstream fuzzer grammars must be preserved."

修改前中文：总体来说，现有 SQL 翻译方法将单条查询作为翻译单位，而非面向模糊测试的完整测试用例种子迁移；后者需要保留跨语句依赖、持续变化的模式上下文以及与下游模糊测试工具语法的兼容性。

修改后英文："RISE processes one self-contained query or procedural artifact at a time, rather than an ordered test sequence of independently executed statements in which later statements may depend on the state produced by earlier ones. These systems target query migration. In contrast, QTRAN targets oracle transfer for logic-bug detection: it uses dialect knowledge and retrieval to translate a query and regenerate its target-side mutant so that the metamorphic relation is preserved. MORPH adopts related techniques, including structural decomposition, localized translation, and dialect-aware retrieval, but targets fuzzing seed transfer. It focuses on maintaining evolving target-side context across an ordered test sequence and on producing seeds that remain parsable and mutable by downstream fuzzers."

修改后中文：RISE 每次处理一个自包含查询或过程式程序单元，而不是由多条独立执行的语句组成、且后续语句可能依赖前面语句所产生状态的有序测试序列。这些系统主要面向查询迁移。相比之下，QTRAN 面向逻辑漏洞检测中的测试预言机迁移：利用方言知识与检索来翻译查询，并重新生成其目标端变异体，以保留变形关系。MORPH 借鉴了结构分解、局部翻译和方言感知检索等已有技术，但面向的是模糊测试种子迁移。它重点关注在有序测试序列中维护不断演化的目标端上下文，并生成能够被下游模糊测试工具解析和变异的种子。

VIII. Data Availability（数据可用性）

40. 修改 Artifact 链接位置（diff p.13 ）

修改前英文："Artifacts and data are available on anonymous Zenodo [51]."

修改前中文：Artifact 和数据可通过匿名 Zenodo [51] 获取。

修改后英文："Artifacts and data are available in our anonymous repository [52]."

修改后中文：Artifact 和数据可通过我们的匿名代码仓库 [52] 获取。

术语对照与阅读提示

英文术语	推荐中文译法	说明
Execution Validity	执行有效率／执行有效性	指目标 DBMS 上能否无运行时错误执行，不是源目标等价性
Result Equivalence	结果等价性	源端与目标端查询结果一致
State Equivalence	状态等价性	执行后的模式、表内容等数据库状态一致
Forward Read-after-Write (RAW) Dependency	前向读后写依赖	后续语句读取前序语句写入的对象
Evolving Target-side Context	持续演化的目标端上下文	目标端翻译后状态随语句执行顺序更新
Object-state Summary	对象状态摘要	对象存在性、定义、列、类型、约束、DDL 变更
Comment-and-restore	注释并恢复机制	在变异期间暂时注释，执行前恢复
Mutation Handling	变异处理策略	MORPH 变异前修复、SEDAR 注释并恢复
Compositional Baseline	组合式基线	SQL 翻译器与统一的 GRIFFIN 后端组合

说明：以上覆盖了 diff 标记中的主要实质性论述修改、重要数值更新和新增实验。文献编号自动顺延、图表排版和相同术语在多个表格中的重复替换，没有逐处展开。对于少数旧稿跨栏、删除文字和新增文字交叠的句子，英文以有完整语义的原文修改片段呈现，不声称是整段逐字符复原。